The structure is pretty much as described previously. First, you create your empty queue as part of the general setup ❶. Then, you create all your promises for the "ready" signals ❷ and get a `std::shared_future` for the `go` signal ❸. Then, you create the futures you'll use to indicate that the threads have finished ❹. These have to go outside the `try` block so that you can set the `go` signal on an exception without waiting for the test threads to complete (which would deadlock—a deadlock in the test code would be rather less than ideal).

Inside the `try` block you can then start the threads ❺, ❻—you use `std::launch::async` to guarantee that the tasks are each running on their own thread. Note that the use of `std::async` makes your exception-safety task easier than it would be with plain `std::thread` because the destructor for the future will join with the thread. The lambda captures specify that each task will reference the queue and the relevant promise for signaling readiness, while taking a copy of the `ready` future you got from the `go` promise.

As described previously, each task sets its own `ready` signal and then waits for the general `ready` signal before running the actual test code. The main thread does the reverse—waiting for the signals from both threads ❽ before signaling them to start the real test ❾.

Finally, the main thread calls `get()` on the futures from the async calls to wait for the tasks to finish ❿, ⓫ and checks the results. Note that the *pop* task returns the retrieved value through the future ❼, so you can use that to get the result for the assert ⓫.

If an exception is thrown, you set the `go` signal to avoid any chance of a dangling thread and rethrow the exception ⓬. The futures corresponding to the tasks ❹ were declared last, so they'll be destroyed first, and their destructors will wait for the tasks to complete if they haven't already.

Although this seems like quite a lot of boilerplate just to test two simple calls, it's necessary to use something similar in order to have the best